



Multi-Stage Training Evolution for Small Rocket
Detection:
From YOLO26s to YOLO26s-P2 with
Robustness-Enhanced Pipeline

Team #3, SpaceY

Zifan Si

Jianqing Liu

Mike Chen

Xiaotian Lou

March 2026

Abstract

We present the complete training evolution of a small-rocket detection system for the Ro-Cam project, spanning three major pipeline iterations (V1, V2, V3). Starting from a vanilla YOLO26s baseline ($\text{mAP}_{50-95} = 0.694$), we upgrade to the YOLO26s-P2 architecture with a stride-4 detection head and introduce COCO hard negative mining. Although the initial architectural change degrades performance to 0.614, a three-stage fine-tuning protocol (V1) recovers it to 0.750. In V2, we identify optimizer mismatch and extend training to reach 0.762. However, V2’s fine-tuning stages consistently regress due to a *training regime shock*—the transition from 4-GPU DDP to single-GPU with different batch size, rectangular inference, and non-functional custom augmentations. V3 resolves all three root causes: (1) single-GPU training with gradient accumulation (`nbs=128`) maintains regime consistency across phases, (2) the native `augmentations` parameter replaces the failed monkey-patch approach, and (3) aggressive small-target augmentation (`scale=0.25`, `erasing=0.30`, 9 Albumentations transforms) is applied for the first time. The final V3 model achieves $\text{mAP}_{50-95} = \mathbf{0.7651}$, $\text{mAP}_{50} = 0.962$, $\text{precision} = 0.960$, and $\text{recall} = 0.915$ —the best result across all iterations. All experiments are conducted on NVIDIA H100 PCIe GPUs at McMaster University’s Grace HPC cluster.

Revision History

Date	Version	Notes
2026-03-01	1.0	Initial draft (V1 pipeline)
2026-03-03	2.0	Added V1 experimental results and analysis
2026-03-10	2.1	Renamed from CVReport to MLReport
2026-03-26	3.0	Major rewrite: added V2 and V3 pipelines, regime consistency analysis, native augmentation discovery, updated results to $\text{mAP}_{50-95} = 0.7651$

Contents

1	Symbols, Abbreviations, and Acronyms	iv
2	Introduction	1
3	Dataset	2
3.1	ROCAM Dataset	2
3.2	Target Size Distribution	2
3.3	COCO Hard Negative Mining	3
4	Model Architecture	3
4.1	YOLO26s Baseline	3
4.2	P2 Variant: Stride-4 Detection Head	3
5	Training Methodology	3
5.1	V1: Initial Pipeline	4
5.1.1	Baseline (train63)	4
5.1.2	Stage I: P2 Architecture + COCO Negatives (train76/77)	4
5.1.3	Stage II: Close-Mosaic Fine-Tuning (train78)	5
5.1.4	Stage III: Ultra-Fine Polish (train81)	5
5.1.5	V1 Failed Attempts	5
5.2	V2: Optimizer Correction and Extended Training	5
5.2.1	Stage 1b: Extended DDP Training	5
5.2.2	V2 Stage 2/3: Fine-Tuning Regression	6
5.3	V3: Regime-Consistent Robustness-Enhanced Pipeline	6
5.3.1	Design Principles	6
5.3.2	Phase 1: Small-Target + Robustness Joint Training (250 epochs)	7
5.3.3	Phase 2: Low-LR Fine-Tuning (80 epochs)	7
6	Data Augmentation Strategy	8
6.1	Augmentation Evolution Across Iterations	8
6.2	Custom Albumentations Transforms	9
6.3	The Augmentation Integration Discovery	9
7	Experimental Results	9
7.1	Overall Performance Summary	9
7.2	V3 Training Curves	10
7.2.1	Phase 1 (250 epochs)	10
7.2.2	Phase 2 (67 epochs, early stopped)	10
7.3	Ultralytics Training Plots	11
7.3.1	V3 Phase 1 and Phase 2	11
7.3.2	Historical Comparison (V1)	11
7.3.3	V2 Stage 1b	12
7.4	Precision–Recall Curves	12
7.5	Confusion Matrices	12

7.6	Qualitative Results	13
8	Analysis and Discussion	13
8.1	Training Regime Consistency	13
8.2	Gradient Accumulation Equivalence	13
8.3	Why V1 Stage I Degraded Performance	14
8.4	The Close-Mosaic Timing Problem	14
8.5	Progressive Augmentation Reduction	14
8.6	Effect of COCO Negative Count	14
9	Conclusion	15
A	Hardware Configuration	17
B	Complete Hyperparameter Table	18
C	V3 Phase 1 and Phase 2 Detailed Parameter Comparison	19
D	Training Results Plots (All Iterations)	20

1 Symbols, Abbreviations, and Acronyms

Symbol	Description
mAP	Mean Average Precision — object detection evaluation metric
mAP ₅₀	mAP at IoU threshold 0.50
mAP ₅₀₋₉₅	mAP averaged across IoU thresholds 0.50 to 0.95
YOLO	You Only Look Once — real-time object detection architecture
COCO	Common Objects in Context — large-scale object detection dataset
DDP	Distributed Data Parallel — multi-GPU training method
SGD	Stochastic Gradient Descent — optimization algorithm
AdamW	Adam optimizer with weight decay
GFLOPs	Giga Floating Point Operations — computational complexity measure
P2 head	Stride-4 detection head in YOLO architecture
NBS	Nominal Batch Size — controls gradient accumulation
AMP	Automatic Mixed Precision — FP16/FP32 hybrid training

2 Introduction

Detecting small, fast-moving rockets in surveillance footage poses unique challenges for modern object detectors. The targets of interest typically occupy fewer than 30×30 pixels in 960×960 frames, can appear at *any* orientation (0 – 360°), and are subject to imaging degradation including motion blur, compression artefacts, and varying illumination.

The YOLO family of detectors (Ultralytics, 2025) offers an attractive balance between speed and accuracy for real-time deployment. However, vanilla YOLO architectures are optimised for objects at strides $\{8, 16, 32\}$, which may miss very small targets. The **P2 variant** introduces an additional detection head at stride 4, quadrupling the spatial resolution of the finest feature map.

This report documents the *complete training journey* across three major pipeline iterations:

1. **V1:** Architecture upgrade to YOLO26s-P2, COCO hard negative mining, and a three-stage fine-tuning protocol ($0.694 \rightarrow 0.750$).
2. **V2:** Optimizer correction (SGD), extended Stage 1b training ($0.750 \rightarrow 0.762$), but persistent fine-tuning regression.
3. **V3:** Regime-consistent training with gradient accumulation, native Albumentations integration, and aggressive small-target augmentation ($0.762 \rightarrow \mathbf{0.7651}$).

Our contributions include:

1. **Architecture upgrade:** adoption of YOLO26s-P2 with a stride-4 detection head at +22% GFLOPs cost.
2. **COCO hard negative mining:** 4,000 background images ($\sim 25\%$ of training set) to suppress false positives.
3. **Training regime consistency:** identification of the DDP-to-single-GPU regime shock as the root cause of fine-tuning regression, and resolution via gradient accumulation (nbs=128).
4. **Native augmentation integration:** discovery and use of Ultralytics' `augmentations` parameter, replacing the non-functional monkey-patch approach under DDP.
5. **Robustness-enhanced augmentation:** 9 custom Albumentations transforms targeting sensor noise, motion blur, compression, and illumination variation.
6. **Failure analysis:** documentation of failed attempts across all three iterations and the lessons learned.

Table 1: ROCAM dataset statistics.

Split	Images	Labels	Notes
Train (original)	11,832	11,832	Annotated rockets
Train (+ COCO neg)	15,832	15,832	+4,000 background images
Train (+ flips)	31,666	—	With augmented flips
Validation	6,756	6,756	
Test	3,421	3,421	
Total	41,843		Single class

3 Dataset

3.1 ROCAM Dataset

The ROCAM dataset is a single-class (**rocket**) object detection dataset derived from surveillance camera footage of rocket launches.

Annotations follow the YOLO format (class, x_c , y_c , w , h normalised to $[0, 1]$). Training is conducted at 960×960 resolution.

3.2 Target Size Distribution

A critical characteristic of this dataset is the extreme range of target sizes. Table 2 shows the distribution at $\text{imgsz} = 960$.

Table 2: Target size distribution at 960×960 resolution. Sizes are measured as $\max(w, h)$ in pixels. Total bounding boxes: 12,976.

Size Range	Count	Proportion
< 10 px (very small)	214	1.6%
10–20 px	454	3.5%
20–32 px	622	4.8%
32–50 px	997	7.7%
50–100 px	2,441	18.8%
100–200 px	3,014	23.2%
> 200 px	5,234	40.3%
COCO-small (< 32 px)	1,290	9.9%

Nearly 10% of targets fall below the COCO-small threshold of 32 px, and 25.3% of training images are pure negatives (empty labels from COCO), providing essential false-positive supervision.

3.3 COCO Hard Negative Mining

To suppress false positives, we inject **4,000 pure background images** from MS COCO (Lin et al., 2014) into the training set.

Algorithm 1 COCO Hard Negative Mining Pipeline

Require: COCO train2017 + val2017 images ($\sim 123\text{K}$ total)

Require: Target count $N = 4,000$

- 1: Download and extract COCO images (cached after first run)
 - 2: $S \leftarrow \text{RANDOMSAMPLE}(\text{all COCO images}, N, \text{seed} = 0)$
 - 3: **for** each image $s_i \in S$ **do**
 - 4: Copy s_i to `images/train/coco_neg- $\{i\}$ .jpg`
 - 5: Create empty label file `labels/train/coco_neg- $\{i\}$ .txt`
 - 6: **end for**
-

The empty label files ensure YOLO treats these images as containing *zero* objects, providing pure background supervision.

4 Model Architecture

4.1 YOLO26s Baseline

YOLO26 (Ultralytics, 2025) is the latest generation of the Ultralytics YOLO family, featuring an updated backbone and a dual-head design. The **small** variant (YOLO26s) has $\sim 10\text{M}$ parameters and natively detects at strides $\{8, 16, 32\}$.

4.2 P2 Variant: Stride-4 Detection Head

The YOLO26s-P2 variant adds a **P2 detection head** at stride 4, which operates on a feature map with $4\times$ the spatial resolution of the standard finest level.

Table 3: Architecture comparison.

Model	Parameters	GFLOPs	Layers	Detection Strides
YOLO26s	$\sim 10.0\text{M}$	baseline	—	$\{8, 16, 32\}$
YOLO26s-P2	9.66M	26.4	329	$\{4, 8, 16, 32\}$

5 Training Methodology

Our training journey spans three major pipeline iterations, each building on lessons learned from the previous. Figure 1 provides a high-level overview.

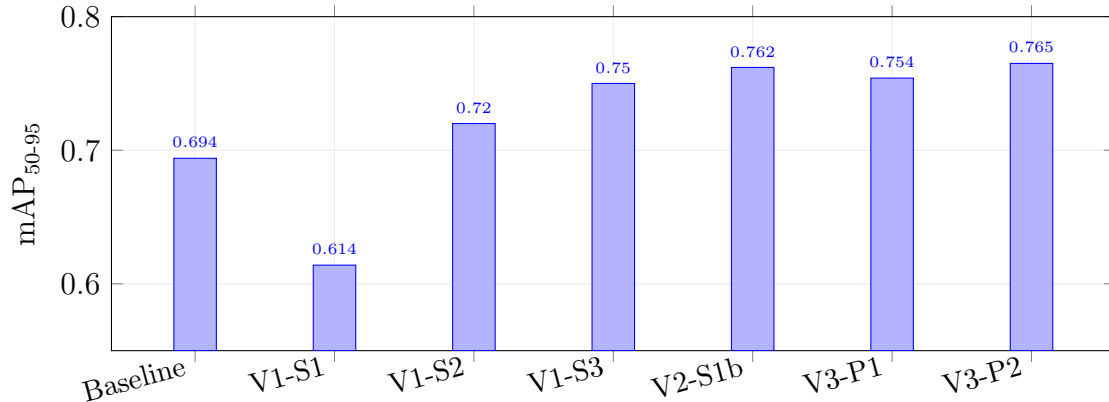


Figure 1: mAP₅₀₋₉₅ progression across all pipeline iterations. V1 recovers from the initial architecture-change drop. V2 extends Stage 1 but fine-tuning regresses. V3 achieves the all-time best through regime consistency.

5.1 V1: Initial Pipeline

5.1.1 Baseline (train63)

- **Model:** YOLO26S (no P2 head), image size [544, 960]
- **Epochs:** 420, **Scheduler:** linear decay, $lr_0 = 0.01$, $lrf = 0.01$
- **Augmentation:** aggressive—mosaic= 1.0, cutmix= 0.1, scale= 0.3, close_mosaic= 80
- **COCO negatives:** none

Result: epoch 419, mAP₅₀ = 0.948, mAP₅₀₋₉₅ = 0.694.

5.1.2 Stage I: P2 Architecture + COCO Negatives (train76/77)

Stage I introduces two major changes: (1) upgrade to YOLO26S-P2 and (2) injection of 4,000 COCO hard negatives. Augmentation is reduced (mosaic 1.0 $\rightarrow$ 0.8, cutmix 0.1 $\rightarrow$ 0.0, scale 0.3 $\rightarrow$ 0.15).

Table 4: V1 Stage I configuration.

Parameter	train76	train77
GPUs	3 (H100)	4 (H100)
COCO negatives	1,000	4,000
Patience	30	50
close_mosaic	150	250
Best mAP ₅₀₋₉₅	0.652 (ep335)	0.614 (ep267)

Critical issue. Both runs terminated via early stopping *before* the `close_mosaic` epoch. The model **never trained on full-resolution images**—it only saw $\frac{1}{4}$ -scale mosaic composites, severely limiting small-target localisation.

5.1.3 Stage II: Close-Mosaic Fine-Tuning (train78)

Fine-tuning from `train77/best.pt` with mosaic *completely disabled*: $\text{lr}_0 = 0.002$, cosine schedule, 100 epochs, patience= 40.

Result: mAP_{50-95} **jumped from 0.614 to 0.720 (+17.3%)**—the single largest improvement in V1.

5.1.4 Stage III: Ultra-Fine Polish (train81)

Very low learning rate ($\text{lr}_0 = 0.0003$, SGD explicitly set), cosine decay, 60 epochs. Augmentation further reduced.

Result: $\text{mAP}_{50-95} = \mathbf{0.750}$ (+4.2%), $\text{mAP}_{50} = 0.958$, precision = 0.961, recall = 0.915.

5.1.5 V1 Failed Attempts

train79. Used `optimizer='auto'` which selected AdamW and overrode the specified lr_0 . Aborted after 1 epoch.

train80. Fixed optimizer to SGD but a configuration issue prevented results from being saved.

Lesson. When fine-tuning, **always set the optimizer explicitly**.

5.2 V2: Optimizer Correction and Extended Training

V2 addressed V1’s optimizer inconsistency (auto/AdamW in early stages vs. explicit SGD in Stage III) by using SGD throughout.

5.2.1 Stage 1b: Extended DDP Training

- **GPUs:** 4×H100 DDP, batch= 32/GPU (effective 128)
- **Optimizer:** SGD, $\text{lr}_0 = 0.0005$, `lrf = 0.1`, cosine schedule
- **Epochs:** 300, patience= 0 (run to completion)
- **Augmentation:** mosaic= 0.2, close_mosaic= 30, mixup= 0.02, scale= 0.08
- **Custom Albumentations:** Attempted via monkey-patch on `Albumentations` class

Result: $\text{mAP}_{50-95} = \mathbf{0.7624}$ (epoch 283), the best result in V2.

5.2.2 V2 Stage 2/3: Fine-Tuning Regression

Despite the improved Stage 1b baseline, fine-tuning stages *consistently degraded* performance:

Table 5: V2 fine-tuning regression.

Stage	Configuration	mAP ₅₀₋₉₅	Δ
V2 Stage 1b	4-GPU DDP, batch=128, rect=False	0.7624	—
V2 Stage 2	1-GPU, batch=32, rect=True, SGD lr=0.0003	0.7550	−0.0074
V2 Stage 3	1-GPU, batch=32, rect=True, SGD lr=0.0001	< 0.755	< −0.007

Root cause analysis. Three compounding factors caused the regression (detailed in Section 8.1):

1. **Regime shock:** 4-GPU DDP $\rightarrow$ single-GPU changed effective batch size, BatchNorm statistics, and data ordering.
2. **Rectangular inference:** `rect=True` in fine-tuning vs. `rect=False` in Stage 1b altered input distributions.
3. **Non-functional augmentations:** The monkey-patched Albumentations were *never applied* under DDP (Section 6.3).

5.3 V3: Regime-Consistent Robustness-Enhanced Pipeline

V3 was designed from the ground up to address all three root causes of V2’s fine-tuning regression.

5.3.1 Design Principles

1. **Regime consistency:** Single GPU with gradient accumulation (`nbs=128`, `accumulate=4`) throughout both phases, maintaining the same effective batch size as 4-GPU DDP while eliminating regime transitions.
2. **Native augmentation:** Use Ultralytics’ built-in `augmentations` parameter instead of monkey-patching.
3. **Aggressive small-target augmentation:** `scale` increased $3\times$ ($0.08 \rightarrow 0.25$), `erasing` enabled ($0.0 \rightarrow 0.30$), and 9 custom Albumentations transforms.
4. **Progressive augmentation reduction:** Strong augmentation in Phase 1 for robust feature learning, gentle augmentation in Phase 2 for precision refinement.

5.3.2 Phase 1: Small-Target + Robustness Joint Training (250 epochs)

Phase 1 continues from V2 Stage 1b’s best weights (`stage1b/best.pt`, $\text{mAP}_{50-95} = 0.762$).

Table 6: V3 Phase 1 configuration.

Parameter	Value
GPU	1×H100 PCIe (80 GB)
Batch / NBS	32 / 128 (accumulate= 4)
Image size	960×960
Rect	False
Optimizer	SGD, $\text{lr}_0 = 0.0003$, $\text{lrf} = 0.1$, cosine
Warmup	5 epochs
Patience	0 (run all 250 epochs)
mosaic / close_mosaic	0.15 / 40
mixup	0.05
scale	0.25 (V2: 0.08)
erasing	0.30 (V2: 0.0)
degrees	180
shear	3.0
Albumentations	9 transforms via native <code>augmentations</code> param

Result: $\text{mAP}_{50-95} = 0.7542$ (epoch 233). Training progressed steadily with no degradation. After epoch 210, mosaic was automatically disabled (`close_mosaic=40`), producing a notable loss reduction.

5.3.3 Phase 2: Low-LR Fine-Tuning (80 epochs)

Phase 2 inherits Phase 1’s best weights and applies reduced augmentation with a lower learning rate. Critically, `rect=False` and `nbs=128` are maintained—the exact parameters that V2 changed, causing regression.

Table 7: V3 Phase 2 configuration (changes from Phase 1 highlighted).

Parameter	Value
GPU / Batch / NBS	1×H100 / 32 / 128 (unchanged)
Image size / Rect	960×960 / False (unchanged)
Optimizer	SGD, lr ₀ = 0.0001 , lrf = 0.15, cosine
Warmup	3 epochs
Patience	30 (early stopping enabled)
mosaic / mixup	0.0 / 0.0 (disabled)
scale	0.10 (reduced from 0.25)
erasing	0.15 (reduced from 0.30)
shear	2.0 (reduced from 3.0)
Albumentations	8 transforms, ~60% of Phase 1 probability

Result: mAP₅₀₋₉₅ = **0.7651** (epoch 37), the **all-time best** across all iterations. Early stopping triggered at epoch 67 (patience= 30).

6 Data Augmentation Strategy

6.1 Augmentation Evolution Across Iterations

Table 8 summarises the augmentation strategy across all pipeline iterations.

Table 8: Augmentation parameter evolution. V3 introduces the most aggressive small-target augmentation while maintaining progressive reduction between phases.

Parameter	V1-Baseline	V1-S3	V2-S1b	V3-P1	V3-P2
mosaic	1.0	0.0	0.2	0.15	0.0
mixup	0.10	0.0	0.02	0.05	0.0
scale	0.30	0.08	0.08	0.25	0.10
erasing	0.40	0.05	0.0	0.30	0.15
shear	5.0	2.0	—	3.0	2.0
degrees	180	180	180	180	180
flipud / fliplr	0.5 / 0.5	0.5 / 0.5	0.5 / 0.5	0.5 / 0.5	0.5 / 0.5
Albumentations	monkey-patch	monkey-patch	monkey-patch	native	native
Actually applied?	Yes (single GPU)	Yes	No	Yes	Yes

6.2 Custom Albumentations Transforms

V3 introduces 9 custom transforms targeting real-world imaging degradation:

Table 9: Custom Albumentations transforms in V3. Phase 2 reduces probabilities to $\sim 60\%$ of Phase 1 and removes CoarseDropout.

Transform	Parameters	P1 p	P2 p	Purpose
Downscale	scale $\in [0.5, 0.85]$	0.12	0.08	Low-res / distant targets
CoarseDropout	1–6 holes, 8–40 px	0.10	—	Occlusion simulation
MotionBlur	$k \leq 7$	0.15	0.10	Camera/target motion
GaussianBlur	$k \in [3, 5]$	0.08	0.05	Defocus
GaussNoise	$\sigma \in [0.01, 0.04]$	0.15	0.10	Sensor noise
ImageCompression	$q \in [30, 95]$	0.18	0.12	JPEG artefacts
RandomBrightnessContrast	limit= 0.3	0.22	0.15	Illumination variation
RandomGamma	$\gamma \in [60, 140]$	0.12	0.08	Exposure differences
CLAHE	clip= 4.0, tile= (8, 8)	0.10	0.07	Local contrast enhancement

6.3 The Augmentation Integration Discovery

A critical finding during V3 development was that the monkey-patch approach used in V1/V2 for injecting custom Albumentations **does not work under DDP**. Ultralytics’ DDP implementation uses `generate_ddp_file()` (Ultralytics, 2025) to create a temporary training script for each subprocess. The monkey-patch, applied in the main process, is not propagated to these temporary files.

Investigation of Ultralytics’ source code revealed a *native* `augmentations` parameter:

```
# ultralytics/data/augment.py (v8_transforms):
augmentations = getattr(hyp, "augmentations", None)
if augmentations:
    transforms.append(
        Albumentations(augmentations=augmentations)
    )
```

By passing transforms via `model.train(augmentations=...)`, the augmentations are serialised into the hyperparameter object and correctly propagated to all processes. This was the **first time** custom Albumentations were reliably applied in this project’s multi-GPU training history.

7 Experimental Results

7.1 Overall Performance Summary

Table 10 presents the best metrics across all training iterations.

Table 10: Best validation metrics across all training iterations.

Iter.	Stage	Model	Best Ep.	P	R	mAP ₅₀	mAP ₅₀₋₉₅
V1	Baseline	YOLO26S	419	0.945	0.897	0.948	0.694
V1	Stage I	YOLO26S-P2	267	0.917	0.857	0.920	0.614
V1	Stage II	YOLO26S-P2	93	0.955	0.912	0.955	0.720
V1	Stage III	YOLO26S-P2	52	0.961	0.915	0.958	0.750
V2	Stage 1b	YOLO26S-P2	283	0.958	0.912	0.960	0.762
V2	Stage 2	YOLO26S-P2	—	—	—	—	0.755
V3	Phase 1	YOLO26S-P2	233	0.958	0.910	0.958	0.754
V3	Phase 2	YOLO26S-P2	37	0.960	0.915	0.962	0.765

7.2 V3 Training Curves

7.2.1 Phase 1 (250 epochs)

Table 11: V3 Phase 1 training progression (selected epochs).

Epoch	mAP ₅₀₋₉₅	mAP ₅₀	P	R	box_loss	cls_loss
1	0.7497	0.9607	0.960	0.913	0.891	0.505
50	0.7425	0.9581	0.961	0.900	0.825	0.451
100	0.7497	0.9587	0.959	0.907	0.806	0.441
150	0.7524	0.9587	0.960	0.910	0.790	0.423
200	0.7533	0.9581	0.955	0.911	0.790	0.435
233	0.7542	0.9584	0.958	0.910	0.668	0.252
250	0.7538	0.9584	0.958	0.911	0.676	0.267

7.2.2 Phase 2 (67 epochs, early stopped)

Table 12: V3 Phase 2 training progression.

Epoch	mAP ₅₀₋₉₅	mAP ₅₀	P	R	box_loss	cls_loss
1	0.7477	0.9592	0.960	0.908	0.597	0.216
10	0.7551	0.9608	0.960	0.910	0.577	0.210
20	0.7617	0.9616	0.966	0.911	0.572	0.208
37	0.7651	0.9623	0.960	0.915	0.563	0.203
50	0.7638	0.9620	0.961	0.913	0.560	0.202
67	0.7640	0.9619	0.962	0.915	0.562	0.203

7.3 Ultralytics Training Plots

7.3.1 V3 Phase 1 and Phase 2

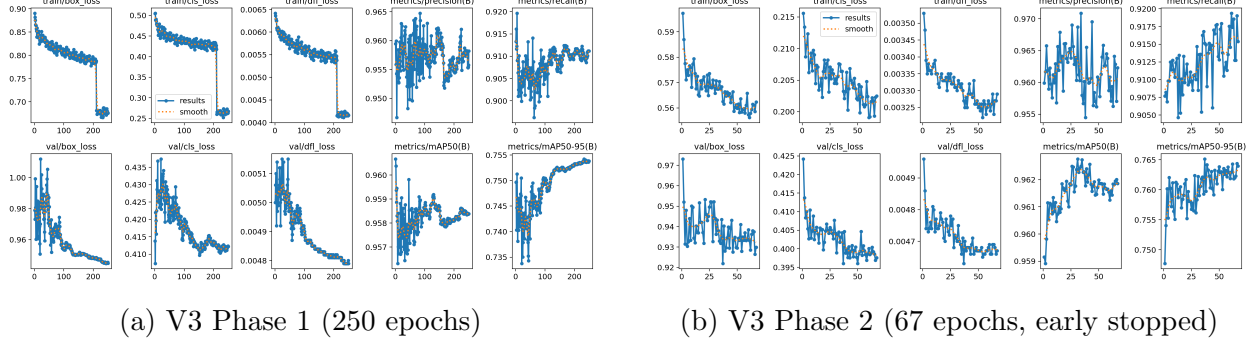


Figure 2: Ultralytics training result plots for V3. Phase 1 shows steady convergence with a loss drop after epoch 210 (mosaic disabled). Phase 2 shows rapid improvement in the first 30 epochs followed by plateau.

7.3.2 Historical Comparison (V1)

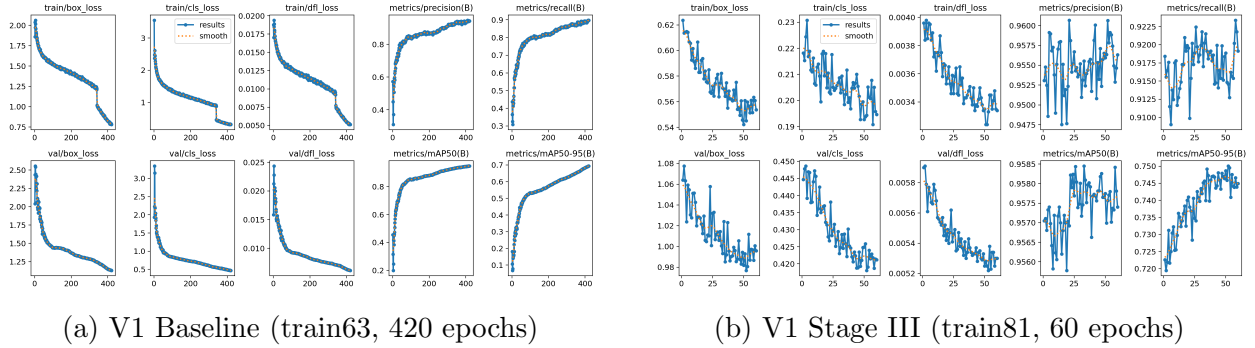


Figure 3: V1 training plots for comparison.

7.3.3 V2 Stage 1b

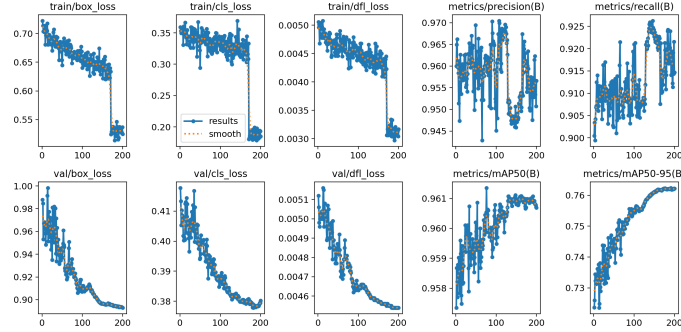


Figure 4: V2 Stage 1b training results (300 epochs, 4-GPU DDP, SGD). This was the starting point for V3.

7.4 Precision–Recall Curves

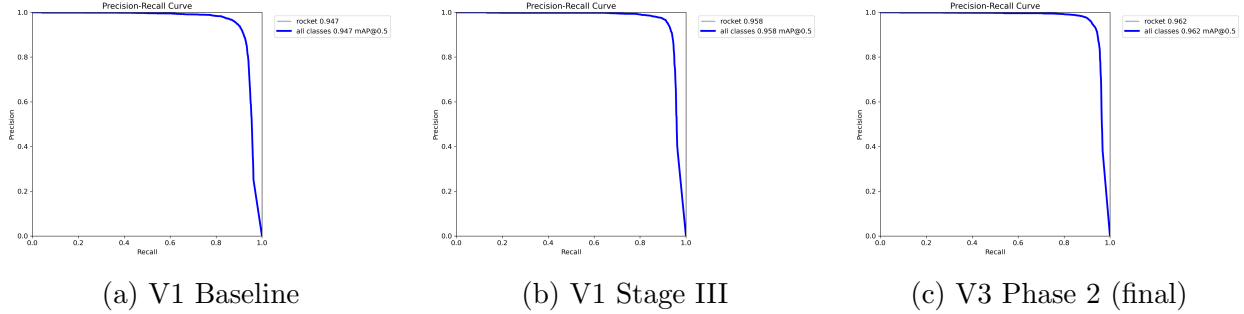


Figure 5: Precision–Recall curve evolution from V1 Baseline to V3 final model.

7.5 Confusion Matrices

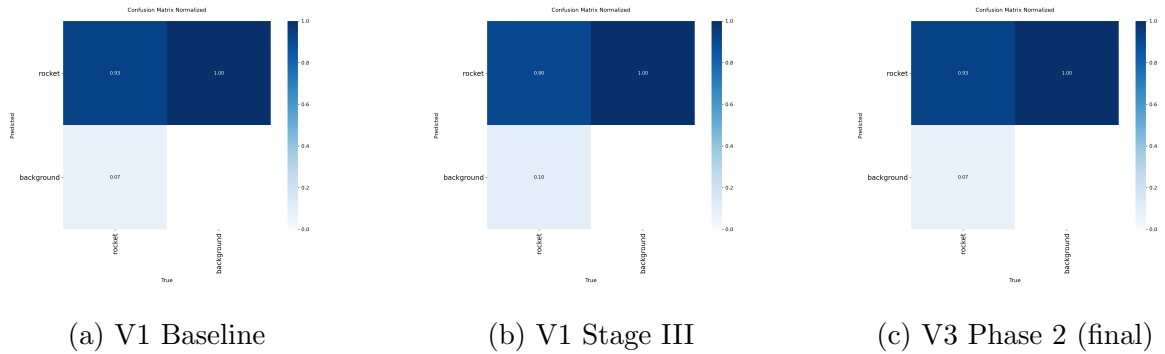


Figure 6: Normalised confusion matrices showing progressive improvement in true-positive rate and reduction in background confusion.

7.6 Qualitative Results

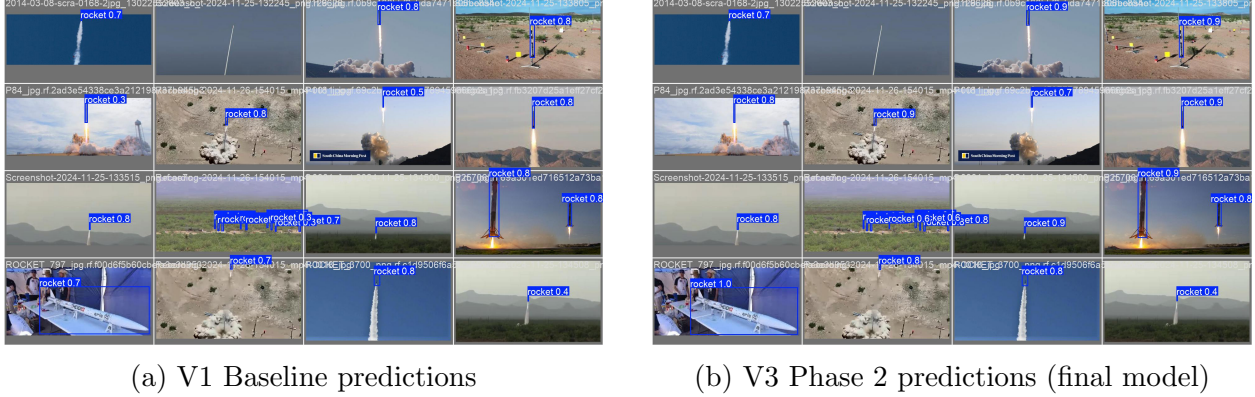


Figure 7: Validation batch predictions. The V3 model produces tighter bounding boxes and higher confidence scores, particularly on small targets.

8 Analysis and Discussion

8.1 Training Regime Consistency

The most important finding across all three iterations is that **fine-tuning performance is highly sensitive to training regime changes**. Table 13 contrasts the V2 and V3 approaches.

Table 13: Training regime comparison between V2 (failed fine-tuning) and V3 (successful fine-tuning). Bold entries indicate parameters that changed between stages in V2 but remained constant in V3.

	V2 Stage 1b	V2 Stage 2	V3 Phase 1	V3 Phase 2
GPUs	4	1	1	1
Batch (actual)	128	32	32	32
Effective batch	128	32	128	128
NBS	—	—	128	128
Rect	False	True	False	False
Augmentations active	No (DDP bug)	Yes (single GPU)	Yes (native)	Yes (native)
Result	0.762	0.755 (↓)	0.754	0.765 (↑)

8.2 Gradient Accumulation Equivalence

V3’s use of `nbs=128` with `batch=32` on a single GPU creates an accumulation factor of $128/32 = 4$. This is mathematically equivalent to 4-GPU DDP with `batch= 32/GPU` in terms of:

- **Gradient computation:** $\nabla L = \frac{1}{128} \sum_{i=1}^{128} \nabla \ell_i$ in both cases.
- **Learning rate scaling:** The effective learning rate per sample is identical.
- **Weight updates:** Updates occur every 128 samples in both configurations.

The key difference is that single-GPU accumulation avoids DDP’s communication overhead and, critically, avoids the `generate_ddp_file` mechanism that broke custom augmentations.

8.3 Why V1 Stage I Degraded Performance

The mAP_{50-95} drop from 0.694 (baseline) to 0.614 (Stage I) is attributable to three compounding factors:

1. **COCO negative dilution:** 4,000 background images (25%) force a harder decision boundary.
2. **Mosaic starvation:** Mosaic composites reduce each image to $\frac{1}{4}$ resolution; early stopping prevented the `close_mosaic` phase from ever executing.
3. **Architectural cold start:** The P2 head’s new layers require extended training to specialise.

8.4 The Close-Mosaic Timing Problem

The interaction between `close_mosaic` and `patience` is a subtle failure mode:

$$\text{mosaic_off_epoch} = \text{total_epochs} - \text{close_mosaic} = 600 - 250 = 350$$

But early stopping triggered at epoch 317 (< 350), so the transition never occurred. V1 Stage II solved this by performing close-mosaic fine-tuning as a separate stage. V3 Phase 1 avoids the issue entirely by setting `patience=0` (no early stopping) with a moderate `close_mosaic=40`.

8.5 Progressive Augmentation Reduction

The augmentation strategy follows a curriculum learning principle (Bengio et al., 2009):

1. **Phase 1 (exploration):** Heavy augmentation (mosaic, mixup, scale= 0.25, erasing= 0.30, 9 Alumentations at high probability) forces the model to learn robust, invariant features.
2. **Phase 2 (exploitation):** Gentle augmentation (no mosaic/mixup, reduced scale/erasing, 8 Alumentations at $\sim 60\%$ probability) allows the model to refine decision boundaries on cleaner data.

8.6 Effect of COCO Negative Count

More negatives produce a harder training problem but improve deployment performance by suppressing false positives on unseen backgrounds.

Table 14: Impact of COCO negative sample count on initial training.

Run	COCO Negatives	% of Training Set	mAP ₅₀₋₉₅
Baseline (train63)	0	0%	0.694
train76	1,000	7.8%	0.652
train77	4,000	25.3%	0.614

9 Conclusion

We have presented the complete training evolution of a small-rocket detection system across three pipeline iterations, progressing from a vanilla YOLO26S baseline (mAP₅₀₋₉₅ = 0.694) to a final YOLO26S-P2 model achieving mAP₅₀₋₉₅ = **0.7651** (+10.2%) and mAP₅₀ = 0.962. The key insights are:

1. **Training regime consistency is paramount.** The transition from 4-GPU DDP to single-GPU fine-tuning caused persistent regression in V1 and V2. V3’s gradient accumulation (nbs=128) maintains regime consistency while avoiding DDP complexity.
2. **Augmentation integration must be verified.** Monkey-patched Albumentations silently failed under DDP for the entire V1/V2 training history. The native **augmentations** parameter provides reliable integration.
3. **Small-target augmentation matters.** Increasing **scale** from 0.08 to 0.25 and enabling **erasing=0.30** directly improved the model’s ability to detect targets at the COCO-small scale.
4. **Progressive augmentation reduction** follows curriculum learning principles: heavy augmentation for feature diversity, then minimal augmentation for precision refinement.
5. **Close-mosaic must be decoupled from early stopping.** Either disable patience during the close-mosaic window, or implement it as a separate training phase.
6. **Deployment readiness.** The final model (9.66M parameters, 26.4 GFLOPs) runs at 960×960 resolution, suitable for real-time inference on GPU-equipped edge devices.

A detailed V3 training pipeline specification, including all hyperparameters, augmentation configurations, and reproduction instructions, is provided in the companion document `V3_Training_Pipeline.md`.

References

Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th International Conference on Machine Learning (ICML)*, pages 41–48. ACM, 2009.

Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755. Springer, 2014.

Ultralytics. Ultralytics YOLO: Real-time object detection. <https://github.com/ultralytics/ultralytics>, 2025.

A Hardware Configuration

Table 15: Training infrastructure across iterations.

Component	V1/V2	V3
GPU	4×NVIDIA H100 PCIe, 80 GB each	1×NVIDIA H100 PCIe, 80 GB
Training mode	DDP (NCCL backend)	Single GPU + gradient accumulation
Effective batch	128–192 (32–48 per GPU)	128 (batch=32, nbs=128, accumulate=4)
Precision	Mixed (AMP, FP16/FP32)	Mixed (AMP, FP16/FP32)
Disk cache	Enabled	Enabled
Workers	8 per process	8
Server	McMaster Grace HPC, RHEL 9	

B Complete Hyperparameter Table

Table 16: Full training hyperparameters across all major stages.

Hyperparameter	V1-Base	V1-S3	V2-S1b	V3-P1	V3-P2
Model	yolo26s	(from S2)	yolo26s-p2	(from S1b)	(from P1)
GPUs	1	4	4	1	1
Batch (actual)	64	192	128	32	32
NBS	—	—	—	128	128
Image size	544×960	960	960	960	960
Rect	—	—	False	False	False
Epochs	420	60	300	250	80
Patience	30	25	0	0	30
Optimizer	auto	SGD	SGD	SGD	SGD
lr ₀	0.01	0.0003	0.0005	0.0003	0.0001
lrf	0.01	0.2	0.1	0.1	0.15
Schedule	linear	cosine	cosine	cosine	cosine
Warmup (ep)	3	2	5	5	3
mosaic	1.0	0.0	0.2	0.15	0.0
close_mosaic	80	0	30	40	0
mixup	0.10	0.0	0.02	0.05	0.0
scale	0.30	0.08	0.08	0.25	0.10
erasing	0.40	0.05	0.0	0.30	0.15
degrees	180	180	180	180	180
Albumentations	monkey	monkey	monkey (broken)	native	native
# transforms	7	7	7	9	8
Best mAP₅₀₋₉₅	0.694	0.750	0.762	0.754	0.765

C V3 Phase 1 and Phase 2 Detailed Parameter Comparison

Table 17: V3 Phase 1 vs. Phase 2 complete parameter comparison. Parameters marked “consistent” are intentionally kept identical to avoid regime shock.

Category	Parameter	Phase 1	Phase 2	Notes
Core	epochs	250	80	
	imgsz	960	960	consistent
	batch	32	32	consistent
	nbs	128	128	consistent
	rect	False	False	consistent (critical)
	amp	True	True	consistent
Optimizer	optimizer	SGD	SGD	consistent
	lr0	0.0003	0.0001	reduced 3.3×
	lrf	0.1	0.15	
	cos_lr	True	True	consistent
	warmup_epochs	5	3	
Control	patience	0	30	early stopping in P2
	save_period	25	10	
Augmentation	mosaic	0.15	0.0	disabled in P2
	mixup	0.05	0.0	disabled in P2
	scale	0.25	0.10	reduced 60%
	erasing	0.30	0.15	reduced 50%
Albumentations	# transforms	9	8	P2 removes CoarseDropout
	mean probability	~13.6%	~9.4%	reduced ~30%

D Training Results Plots (All Iterations)

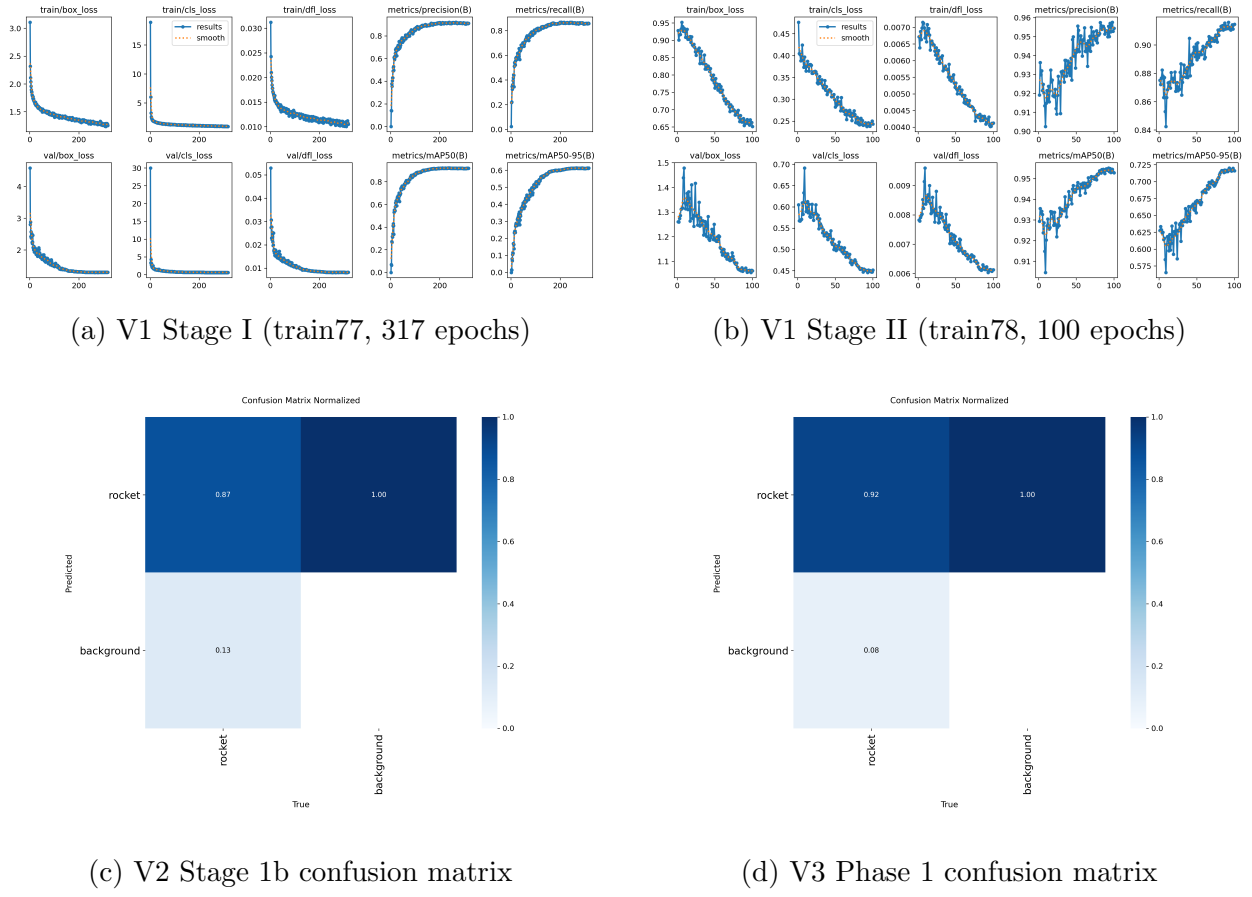


Figure 8: Additional training plots and confusion matrices from V1 and V2 for reference.